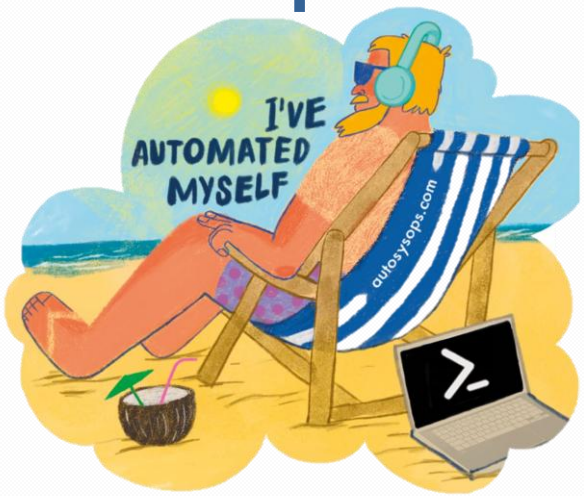
Automation with GitHub Actions and Azure DevOps

Leo Visser



Who am I

- Consultant @ BBTG
  - AI, Automation, Cloud
- DuPSUG Organizer
- Azure MVP (PowerShell)

AUTO-SYS-OPS

Welcome

AUTO-SYS-OPS

Rules:

- No question is stupid
- If you are done feel free to help others

Link:
https://github.com/autosysops/WorkshopAutomation

What are we going to do?

- We start with Azure Devops and end with GitHub
- Building pipelines with different stages/jobs etc
- Dependencies in pipelines
- Variables and Secrets
- Splitting the pipeline up
- Using git commands in pipelines
- What's next

What are we going to do?

- We start with Azure Devops and end with GitHub
- Building pipelines with different stages/jobs etc
- Dependencies in pipelines
- Variables and Secrets
- Splitting the pipeline up
- Using git commands in pipelines
- What's next

What are pipelines and actions

- Automation for CI/CD
- Needed for deployment and building
- Azure DevOps → GitHub
- Low-Code Pipelines → YAML Pipelines
- Self Hosted or Provided
  - Scalable hosted version available
- Authentication via PAT tokes

Hierachy

- Stages (ADO)
- Jobs (ADO & GitHub)
- Steps (ADO & GitHub)

Dependencies

- Between jobs or stages, to make sure parallel processing works well
- Jobs run in parallel unless stated otherwise
- Stages run in series unless stated otherwise

Variables and Secrets

- Variable Scoping (global, stage, job, script)
- Variables in:
  - Code (ADO & GitHub)
  - Groups (ADO)
  - Repository Vars (GitHub)
- Variables easy to use, Secrets masked and encrypted

Templates

- Ways to split up logic in the pipelines and reuse it
- Azure DevOps:
  - Allows for free placement and folder structure
  - No special marker needed
  - Choise in how secret scope is handled
- GitHub:
  - Limited to one folder
  - Needs specific trigger
  - Always isolated

Parameters

- Way to add options to your workflows
- Multiple types of parameters available
- When run a form will appear

Compile and Runtime

- Pipelines/Action are compiled with parameter data
- Variables can be in compile or runtime based on how they are used

Passing on info

- Output variables are stored in Runtime and can be retrieved by new jobs and stages
- Commands send as text are interpreted by the pipeline (syntax differences between ADO and GitHub)
- Do keep in mind that dependencies in execution order are set correctly!

Using Git

- Give the pipeline permission to push to the Repo
  - Azure DevOps: special account inside ADO which can be granted permission
  - GitHub: handled in code in the workflow
- Option to push data back to branches
- Checkout will retrieve data for specific git hash so if data is pushed keep in mind for later jobs/stages this hash needs to be updated

Marketplaces

- Azure DevOps and GitHub have a big marketplace for extra kind of steps for things like:
  - Creating and completing pullrequests
  - Connecting to Azure or other platforms
  - Compiling bicep or other types of code
- Marketplace is checked but vulnerable so be careful with extensions

What's next

- Many more possibilities like:
  - Using environments in ADO or GitHub or determine which settings and variables are used more easily
  - Using matrixes to create loops for many different circumstances
  - Publishing reports back to the pipeline run screen to show info about testing etc.

Questions?

Don’t forget to rate in the app

Blog: www.autosysops.com
Socials: @AutoSysOps